Banker's Algorithm

Deadlock Avoidance in Operating Systems

Course: Operating Systems
Roll No: 061, 066, 306
Semester: BSCS-V-B

Introduction to Deadlocks

A state where two or more processes are indefinitely waiting for each other to release resources.

Mutual Exclusion

Only one process can use a resource at a time.

Hold and Wait

Process holds resources while waiting for new ones.

Circular Wait

Processes form a circular chain, each waiting for the next.

No Preemption

Resources cannot be forcibly taken from a process.

Deadlock Handling Approaches

Strategies to manage and mitigate deadlocks in an operating system.

Deadlock Prevention

Negates one of the four necessary conditions.

Deadlock Avoidance

Dynamically checks resource-allocation state.

Deadlock Detection & Recovery

Detects deadlocks and then recovers from them.

Banker's Algorithm is a prominent Deadlock Avoidance technique.

What is Banker's Algorithm?

Ensuring system safety through a resource allocation strategy.

- **Proposed by Edsger Dijkstra:** A pioneering concept in OS resource management.
- **Banking System Analogy:** Named for its parallel to how banks ensure they can meet all customer demands.
- **Never Enters Unsafe State:** Guarantees that the system can always allocate resources to all processes in some order and avoid deadlock.
- **Known Max Resource Needs:** Requires processes to declare their maximum resource requirements upfront.

Safe State

A state where the system can allocate resources to each process (up to its maximum) in some order and still avoid deadlock.

Unsafe State

A state that might lead to a deadlock; there is no sequence of resource allocation that guarantees all processes can finish.

Key Data Structures

Fundamental matrices and vectors used in the Banker's Algorithm.

Available	A vector of length 'm' indicating the number of available resources of each type. (e.g., A[j] = k, k instances of resource type j are available)
Max	An n x m matrix defining the maximum demand of each process. Max[i,j] = k means Process P[i] may request at most k instances of resource type R[j].
Allocation	An n x m matrix defining the number of resources of each type currently allocated to each process. Allocation[i,j] = k means Process P[i] is currently allocated k instances of R[j].
Need	An n x m matrix indicating the remaining resource need of each process. Need[i,j] = k means Process P[i] needs k more instances of R[j] to complete its task.

> **Formula:** Need[i][j] = Max[i][j] – Allocation[i][j]

Safety Algorithm (Core Logic)

Determining if the system is in a safe state.

Initialize | Find Process | Allocate | Mark Finish | Repeat

This algorithm iteratively checks if a safe sequence of processes exists, ensuring all processes can complete without deadlock.

Resource Request Algorithm

Check Request ≤ Need

Is the request within the process's declared maximum need

Check Request ≤ Available

Are there enough resources currently available in the system?

Pretend Allocation

Temporarily allocate resources to simulate the new state..

Run Safety Algorithm

Execute the safety algorithm on this simulated state.

If Safe → Grant

If the simulated state is safe, commit the allocation.

Else → Rollback

If unsafe, deny the request and restore the previous state.

Worked Example

Initial State

- Processes: P0, P1, P2
- Resources: A, B, C

Available Resources: 3, 3, 2

	Allocation	Max	Need
P0	[A=0, B=1, C=0]	[A=7, B=5, C=3]	[A=7, B=4, C=3]
P1	[A=2, B=0, C=0]	[A=3, B=2, C=2]	[A=1, B=2, C=2]
P2	[A=3, B=0, C=2]	[A=9, B=0, C=2]	[A=6, B=0, C=0]

Safe Sequence Result

The algorithm determines a safe sequence, for example:

System is in a Safe State.

1

P1

Can finish as Need[P1] ≤ Available

2

P0

Can finish after P1 releases resources

3

P2

Can finish after P0 releases resources

Advantages, Limitations & Conclusion

Advantages

- Effectively prevents deadlocks.
- Ensures the system remains in a safe state.
- Provides predictable resource management.

Limitations

- Requires prior knowledge of maximum resource needs.
- Incurs high computational overhead.
- Not suitable for highly dynamic systems.

The Banker's Algorithm is a powerful tool for deadlock avoidance, ensuring system stability by meticulously checking for a safe sequence before any resource allocation.

```python
# Available resources
available = [3, 3, 2]

# Maximum matrix
maximum = [
    [7, 5, 3],
    [3, 2, 2],
    [4, 3, 3]
]

# Allocation matrix
allocation = [
    [0, 1, 0],
    [2, 0, 0],
    [0, 0, 2]
]

# ------------------------------
# Calculate Need Matrix
# ------------------------------
def calculate_need(maximum, allocation):
    need = []
    for i in range(processes):
        row = []
        for j in range(resources):
            row.append(maximum[i][j] - allocati
        need.append(row)
    return need
```

```python
def is_safe():
    finish = [False] * processes
    safe_sequence = []
    work = available.copy()

    print("\n--- ALLOCATION MATRIX ---")
    for i in range(processes):
        print(f"P{i}: {allocation[i]}")

    print("\n--- MAXIMUM MATRIX ---")
    for i in range(processes):
        print(f"P{i}: {maximum[i]}")

    print("\n--- NEED MATRIX ---")
    for i in range(processes):
        print(f"P{i}: {need[i]}")

    print("\n--- BANKER'S ALGORITHM EXECUTION ---")

    while len(safe_sequence) < processes:
        found = False
        for i in range(processes):
            if not finish[i]:
                if all(need[i][j] <= work[j] for j in
                    print(f"\nProcess P{i} is executin
                    for j in range(resources):
                        work[j] += allocation[i][j]
                    safe_sequence.append(i)
                    finish[i] = True
                    found = True
                    print(f"Available Resources Now: {
```

```python
s_safe():

rint("\n--- NEED MATRIX ---")
or i in range(processes):
    print(f"P{i}: {need[i]}")

rint("\n--- BANKER'S ALGORITHM EXECUTION ---")

hile len(safe_sequence) < processes:
    found = False
    for i in range(processes):
        if not finish[i]:
            if all(need[i][j] <= work[j] for j in range(
                print(f"\nProcess P{i} is executing...")
                for j in range(resources):
                    work[j] += allocation[i][j]
                safe_sequence.append(i)
                finish[i] = True
                found = True
                print(f"Available Resources Now: {work}"

    if not found:
        print("\n❌ SYSTEM IS NOT IN SAFE STATE (Deadlo
        return

rint("\n✅ SYSTEM IS IN SAFE STATE")
rint("SAFE SEQUENCE:", " -> ".join(f"P{i}" for i in saf
```

Algorithm

fe()

```
==============================================================
        BANKER'S ALGORITHM SIMULATION PROGRAM
==============================================================

--- ALLOCATION MATRIX ---
P0: [0, 1, 0]
P1: [2, 0, 0]
P2: [3, 0, 2]
P3: [2, 1, 1]
P4: [0, 0, 2]

--- MAXIMUM MATRIX ---
P0: [7, 5, 3]
P1: [3, 2, 2]
P2: [9, 0, 2]
P3: [2, 2, 2]
P4: [4, 3, 3]

--- NEED MATRIX ---
P0: [7, 4, 3]
P1: [1, 2, 2]
P2: [6, 0, 0]
P3: [0, 1, 1]
P4: [4, 3, 1]

--- BANKER'S ALGORITHM EXECUTION ---

Process P1 is executing...
Available Resources Now: [5, 3, 2]

Process P3 is executing...
Available Resources Now: [7, 4, 3]

Process P4 is executing...
Available Resources Now: [7, 4, 5]
```

main ⊗ 0 ⚠ 0 Watch

```
--- BANKER'S ALGORITHM EXECUTION ---

Process P1 is executing...
Available Resources Now: [5, 3, 2]

Process P3 is executing...
Available Resources Now: [7, 4, 3]

Process P4 is executing...
Available Resources Now: [7, 4, 5]

Process P0 is executing...
Available Resources Now: [7, 5, 5]

Process P2 is executing...
Available Resources Now: [10, 5, 7]

✅ SYSTEM IS IN SAFE STATE
SAFE SEQUENCE: P1 -> P3 -> P4 -> P0 -> P2
```